Exemplo

Programação de algoritmos paralelizáveis com *threads*

Cálculo do valor de PI com o método Monte Carlo

(demonstração do impacto na performance e na quantidade de trabalho realizada)

Este exemplo:

- **Utiliza várias *threads* para calcular o valor de PI**
  - **O valor de PI, pelo método Monte Carlo, é obtido pela razão entre o número total de dardos que são atirados sobre um quadrado e aqueles que caem sobre um círculo inscrito nesse quadrado**
    - Os dados são simulados sorteando as suas coordenadas x e y
    - Métodos de monte Carlo: https://en.wikipedia.org/wiki/Monte_Carlo_method
  - **Quantos mais dardos, mais casas decimais estarão corretas**
    - Desde que o gerador de número aleatórios seja, de facto, uniforme
  - **É necessário um número muito grande de dardos**
  - **Para otimizar a performance, lançam-se várias *threads* (número parametrizável)**
    - Cada *thread* lança ("simula") dardos
    - No final, usam-se todos os dardos lançados por todas as *threads*
  - **Enquanto o número de *threads* não ultrapassar o número de núcleos disponíveis ganha-se performance por cada nova *thread* utilizada**
    - **Isto é visível experimentalmente:**
      - a performance (número total de dardos lançados na mesma quantidade de tempo)
      - a utilização intensiva dos núcleos de processador, tantos quantas as *threads* em uso (programa **htop**)
      - Presume-se que o sistema não está a ocupar núcleos cm outras coisas alheias a este programa
    - **Se as *threads* ultrapassarem o número de núcleos disponíveis, então já não se ganha performance, eventualmente até se perde (maior competição entre as *threads*)**

Estratégia de implementação usada:

- Todas as *threads* são baseadas na mesma função (é o mesmo algoritmo, apenas os dados variam)
    - É passado um ponteiro para as *threads* de forma a que cada uma saiba o que deve fazer (estratégia habitual)

- A *thread* inicial, a correr a função main, lança as *threads* e depois aguarda um intervalo de tempo. Findo esse intervalo indica às *threads* que devem terminar
    - O número de *threads* é indicado em linha de comandos
    - O intervalo de tempo é indicado em linha de comandos (segundos)
    - As *threads* são terminadas pela estratégia habitual "variável continua"

- A função main aguarda que as *threads* tenham garantidamente terminado, e só então é que vai verificar os totais de dardos que cada thread produziu
    - pthread_join

- OS computadores habituais são suficientemente rápidos para fazerem convergir o valor de PI para um número aproximado muito rapidamente, dificultando as experiências em que se faz variar o número de *threads* e o tempo de trabalho para observar a qualidade do valor de PI obtido. Assim, o programa permite perder tempo artificialmente executando contas sem significado em ciclo (evita-se o uso de *sleeps* porque se pretende observar a ocupação dos núcleos, coisa que o uso de *sleep* impediria). Este atraso artificial é designado de fator "slowdown"
    - O fator *slowdown* é indicado em linha de comandos. O valor 0 significa sem atraso nenhum. Um valor de 10 corresponde a um atraso já bastante considerável pelo que se recomenda o uso de valores pequenos.

Ficheiros *header* envolvidos

```
#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#include <pthread.h>
#include <unistd.h>
#include <locale.h>
```

Protótipo para a função da thread

```
void * calculaPI(void *);
```

Estrutura de dados para controlar as threads

```
typedef struct {
   pthread_t tid;
   int continua;
   unsigned long long int total, dentro;
   int slowdown;
   int seed;
} TDADOS;
```

Constantes simbólica: número máximo de threads

```
#define MAXIMUMT 20
```

Função main

1. **Obtém a configuração de tempo e número de threads**
2. **Prepara e lança as threads**
3. **Aguarda o tempo especificado**
4. **Indica às threads para terminarem findo o tempo especificado**
5. **Aguarda que as threads realmente tenham terminado**
6. **Recolhe a informação de dardos lançados e dardos dentro do círculo**
7. **Apresenta o valor de PI e termina**

```
int main(int argc, char * argv[]) {
   int NTHREADS, NSECS, SLOWDOWN;  // não são constantes simbólicas
   TDADOS workers[MAXIMUMT];
   int i;
   unsigned long long int allDentro = 0, allTotal = 0;
   long double allPI = 0;
   long double localPI;
```

1 - Obtém dados da linha de comandos: intervalo de tempo e número de threads a usar

```
   if (argc<4) {
       printf("\n%s segundos threads slowdown (0=nenhum)\n", argv[0]);
      return 1;
   }

   NSECS = atoi(argv[1]);      // estas conversões
   NTHREADS = atoi(argv[2]);   // deviam ser todas verificadas
   SLOWDOWN = atoi(argv[3]);   // (fica para TPC)
   // Estes valores deviam ser sanitizados (-> TPC tb)
```

```
printf("\nNum segundos = %d", NSECS);
printf("\nNum Threads = %d", NTHREADS);
printf("\nFactor slowdown = %d", SLOWDOWN);
printf("\n");
```

2 – Prepara e lança as threads

```
for (i = 0; i< NTHREADS; i++) {
    workers[i].continua = 1;
    workers[i].seed = i;  // cada thread tem um i (seed) diferente
    workers[i].slowdown = SLOWDOWN;
    // restantes dados são inicializados na thread
    pthread_create(& workers[i].tid, NULL, calculaPI, workers + i);
}
```

3 - Aguarda o tempo especificado

```
// aguarda N segundos
printf("\nA aguardar %d segundos\n", NSECS);
sleep(NSECS);
```

4 - Indica às threads para terminarem findo o tempo especificado

```
// informa threads devem parar;
for (i=0; i< NTHREADS; ++i)
   workers[i].continua = 0;
printf("\nThreads informadas para terminar");
```

5 e 6 - Aguarda que as threads realmente tenham terminado, obtendo os valores produzidos

```
// aguarda threads realmente terminarem e recolhe resultados
for (i=0; i<NTHREADS; ++i) {
   pthread_join(workers[i].tid, NULL);
   localPI = (long double) 4.0 * workers[i].dentro /
                                                workers[i].total;
   printf("\n%d -> dentro = %llu, total = %llu, PI = %Lf",
         i,
         workers[i].dentro, workers[i].total,
         localPI);
   allDentro += workers[i].dentro;
   allTotal += workers[i].total;
}
```

7 - Apresenta o valor de PI e termina

```
   allPI = (long double) 4.0 * allDentro / allTotal;
   printf("\n\n PI global = %Lf", allPI);
   setlocale(LC_NUMERIC, "");
   printf("\n\n Dardos total = %llu", allTotal);
   printf("\n\n");
   return 0;
}
```

Função da thread

Encontra-se em ciclo (controlado pela estratégia "variável continua") sorteando dardos. No final passa os valores totais para variáveis que a função main poderá consultar

srand_r() e **drand_r** --> ***_r*** indica que é uma função thread safe. ***drand48*** segue uma distribuição uniforme melhor que srand() / *rand()* e é *thread* safe.

IMPORTANTE: o gerador de números aleatórios é inicializado com base no tempo atual. A diferença de tempo da criação de cada *thread* fará todas as *threads* passarem pela inicialização em tempos demasiado próximos para haver diferença na chamada à função *time()*. Assim, usa-se um inteiro passado a cada *thread* (diferente em cada uma) para distinguir a *seed* na inicialização do gerador de números aleatórios (campo "seed" na estrutura de dados da *thread*)

A função da *thread* simula uma execução mais lenta, para melhor observabilidade das experiências que possam ser feitas com este programa, invocando a função *simulaMaisLento*, cujo protótipo é declarado antes para ser reconhecido pelo compilador.

```
// protótipo da função simulaMaisLento (feita mais adiante)

void simulaMaisLento();


// função da thread

void * calculaPI(void * p) {
   TDADOS * mydados = (TDADOS *) p;
   double x,y;
   unsigned long long int dentro = 0, total = 0;
   // inicializa randomize (rand e rand_r estão deprecated)
   struct drand48_data randBuffer;
   srand48_r(time(NULL) + mydados->seed * 123000, &randBuffer);
```

```
   while (mydados->continua) {
      drand48_r(&randBuffer, &x);  // 0 <= x >= 1
      drand48_r(&randBuffer, &y);  // 0 <= y >= 1
      // if (sqrt(x*x + y*y) <= sqrt(1)) ... sqrt pode ser omitido
      if (x*x + y*y <= 1)
         ++dentro;
      ++total;
      // simula lento
      for (int i = 0; i< mydados->slowdown; ++i)
         simulaMaisLento();
   }
   mydados->total = total;
   mydados->dentro = dentro;
   return NULL;
}
```

Função da simulaMaisLento

Executa algumas contas sem sentido apenas para perder algum tempo e mantendo o processador ocupado. A ocupação do processador representa e simboliza uma operação que, noutro cenário menos simples, ocuparia também o processador.

Esta função é chamada a partir da função da *thread* tantas vezes em cada dardo simulado como as que foram especificadas na linha de comandos.

Os valores usados nos ciclos foram ajustados manualmente num contexto de máquina virtual num PC comum.

```
void simulaMaisLento() {
    int a=0, b=0;
    for (int i=1; i<=100; ++i)
       for (int j=1; j<=1000; ++j)
          a += j*i % (j+i);  // contas ao calhas para gastar tempo
    b = a;
}
```

Experiências que se podem fazer usando este programa

- Ocupação de 1 núcleo por thread lançada
- Ganho de qualidade do valor obtido sem custo adicional de tempo enquanto não se esgotam os núcleos disponíveis
- Estabilização da qualidade do valor (para o mesmo tempo) mesmo que se aumentem as *threads* após se terem esgotado os núcleos disponíveis

Importa saber quantos núcleos existem e ver a ocupação destes

- comando **htop** (se não estiver instalado -> sudo apt install htop)

Exemplo:

5 segundos, 4 threads, slowndown 3

```
joao@debian:~/Desktop/aulas/a7$ ./montecarlo 5 4 3

Num segundos = 5
Num Threads = 4
Factor slowdown = 3

A aguardar 5 segundos

Threads informadas para terminar
0 -> dentro = 1338, total = 1704, PI = 3.140845
1 -> dentro = 1337, total = 1674, PI = 3.194743
2 -> dentro = 1325, total = 1684, PI = 3.147268
3 -> dentro = 1284, total = 1666, PI = 3.082833

 PI global = 3.141498

 Dardos total = 6728

joao@debian:~/Desktop/aulas/a7$
```

Ocupação dos núcleos (virtuais) durante a execução da experiência acima (obtido com **htop**)

